

脚手架级微服务租房平台

通用问题

1. 为什么做这个项目？

- 回答1：（出于兴趣爱好开发）

大学期间，我和同学在外合租过一段时间，使用了一些租房平台，于是我有个想法，自己能不能开发一个租房平台，可以让我将理论知识与实践相结合。我希望通过实际项目来加深对 Java 编程语言和相关技术的理解。于是我便查找了一些资料，看了一些开源项目，进行了一些改进。

- 回答2：（开源项目的解释）

这个项目其实是在 github 上找到的一个开源项目，主要是可以支持一些常规的聊天项目，我对于聊天如何实现的比较感兴趣。顺便也想锻炼一下自己工程代码能力，在网上就找到了这个开源项目和项目的一些比较完善的文档和介绍，再加上找了一个业务场景：租房。所以就确定了这个项目。

- 回答3：（学校课设的解释）

这是学校的课设题目，然后在课设项目的基础上，查找了一些资料，进行了一些改进。

2. 这个项目为啥和上个(之前)同学的项目一样？

- 回答1：（开源项目的回答）

因为这个项目本身就是开源项目，网上一搜都是好多都是这个项目的博客分析，写得优质的主要就是那几篇，可能我们参考了一样的博客讲解梳理框架吧。抽奖系统虽然听起来简单，但实际上涉及到数据存储、状态转换、异常处理等多个技术点。

- 回答2：（学校课设的回答）

这个问题面试官可能会直接的指出：上个同学和你学校不同，但是跟你的项目完全一样。

这个项目确实是我们学校的开源项目，但是我在实现的时候也在网上找了跟课设要求类似的开源项目，而且发现网上针对这个类型的项目文档还听完善的，可能其他的同学也是在网上跟我一样找到了类似的博客或开源项目进行借鉴的吧。

（注：以下内容需要同学自己思考项目扩展，如不清楚，不建议参照话术，容易给自己挖坑。说到上面的内容就可以了。不要使用课上讲解的扩展，大家都说一样的也不行）

但这个项目除了常规流程，我还进行了（功能扩展/性能优化/代码优化）.....(并解释一下扩展点)

可以参考的回答思路：即使面试官的指控让你感到不舒服，也要尽量保持冷静和专注。无论情况如何，保持专业和礼貌的态度是非常重要的。

- **诚实面对：** 如果确实使用了代码而没有适当的引用或修改，那么诚实地承认这一点，可以说是从 github 上找的开源项目，并解释你当时的想法和决策过程（参考答案举例）。同时，如果抄袭代码被面试官发现，要及时表明你已经认识到了这个问题，并在未来的工作中会更加注意。
- **展示学习过程：** 如果是面试官发现项目确实有抄袭现象，也可以说项目原本的一些问题，你进行了优化解决，体现自己有过思考，并解释问题与优化思路。
- **请求反馈：** 询问面试官对你的项目有什么具体的反馈或建议，这表明你愿意学习和改进。

3. 介绍一下这个项目。

回答一：（精简版）

基于脚手架的微服务在线租房系统，业务模型对标了贝壳、安居客、闲鱼等流行应用。无论是交互体验、架构设计，还是数据模拟都达到真实的工业水准设计。

本项目是一个 Java 综合项目，涵盖【前端】与【后端】开发，业务上包含房源管理、租户和房东沟通、房源上下架等整套业务流程。核心技术包括微服务、分布式集群、高并发、分布式对象存储、高速缓存、消息队列、即时通信、解决方案设计。

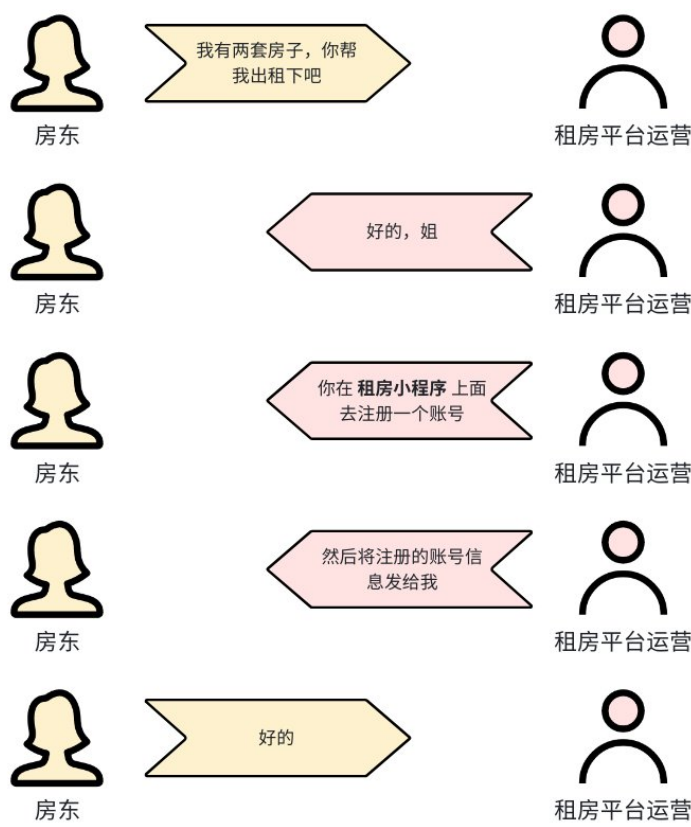
- **后端采用 Spring Cloud 微服务架构**，引入**脚手架**，将系统拆分为**五个微服务**。服务间借助 OpenFeign 实现相互调用。同时，项目集成了 Nginx、Nacos、MySQL 等 10 多个主流 Java 后端组件，技术栈丰富且前沿。
- **前端包含 Web 管理端和微信小程序端**，基于 **Vue 3 框架**，使用 Vite 构建工具，以 JavaScript 为主要语言，利用 Axios 高效交互数据，并引入 Element-Plus 组件库，让系统界面既实用又美观。C 端使用了微信小程序作为面向移动端应用入口。

回答二：（完整版 [根据面试官状态选择回答]）

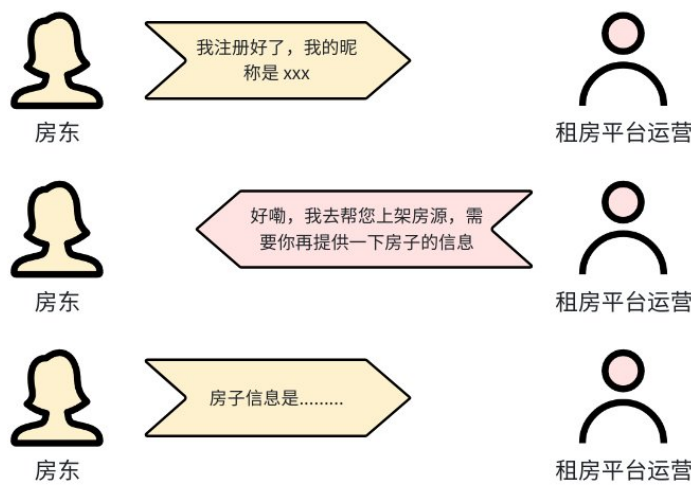
1. **项目背景：** 大学期间，我和同学在外合租过一段时间，使用了一些租房平台，于是我有个想法，自己能不能开发一个租房平台，可以让我将理论知识与实践相结合。我希望通过实际项目来加深对 Java 编程语言和相关技术的理解。于是我便查找了一些资料，看了一些开源项目，进行了一些改进。
2. **项目目标：** 于是我希望提供一个功能全面、操作简便、安全可靠的租房系统。
3. **个人角色：** 由于本项目是一个 Java 综合项目，涵盖【前端】与【后端】开发，我负责后端的主要工作，包含项目的设计，需求的设计，功能的拆分，后端的开发。前端开发由我同学负责。在这个过程中，也是我来对项目节奏进行把控。
4. **项目设计：** 项目我采用了 Spring Cloud 微服务架构。
 - a. 引入脚手架，将系统拆分为五个微服务：包括网关微服务、咨询微服务、首页微服务、基础管理微服务、文件微服务

5. 项目介绍：

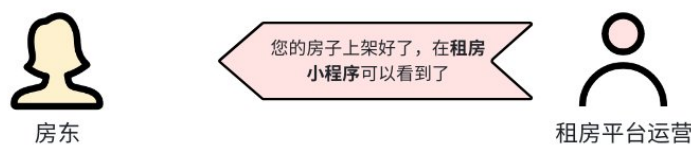
a. 该项目最主要的业务就是租房与咨询。

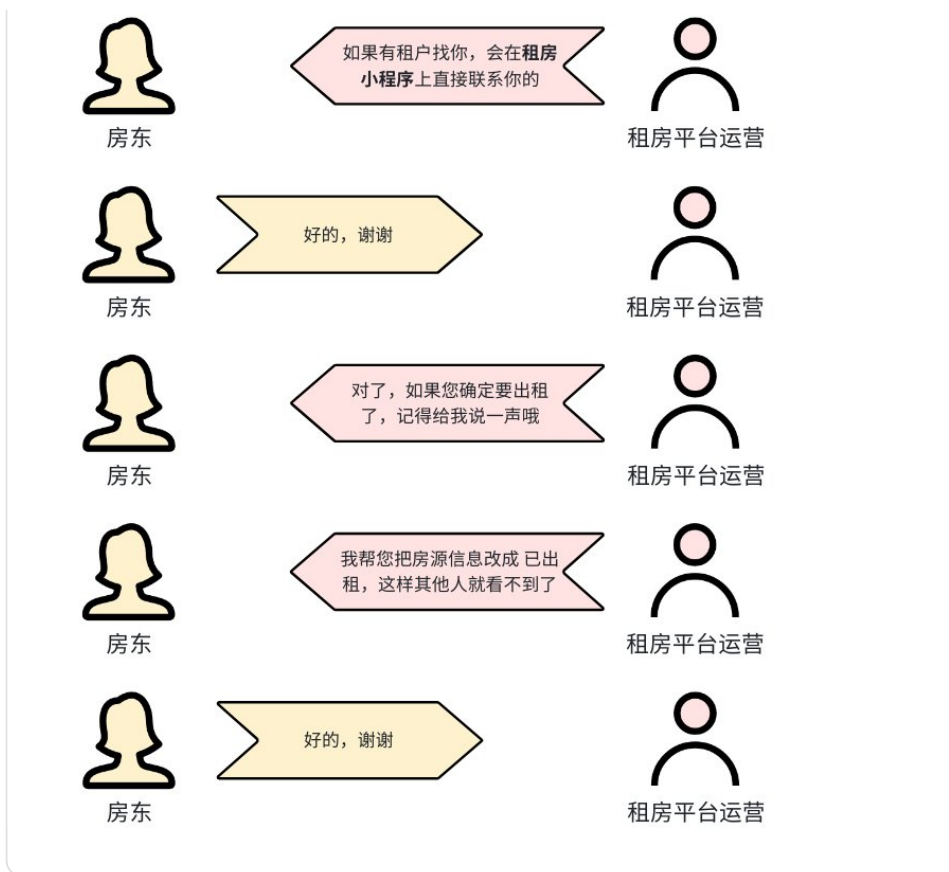


过了一会，房东打开了租房小程序，并且注册了一个新用户。



当天平台运营约了房东去现场实地核实房源真实性。
确认无误后，租房平台运营将房东的房子通过租房管理端成功发布到租房小程序。





6. 技术栈：在项目中，我使用了：

- a. Spring Boot3 + JDK 17
- b. 比特JAVA脚手架：bite-framework-java
- c. 服务注册发现和配置管理中心：Nacos
- d. 数据库：MySQL + MyBatis-Plus
- e. 分布式锁：Redisson
- f. 实时通信协议：WebSocket
- g. 认证方案：JWT
- h. 缓存：Redis
- i. 消息队列：RabbitMQ
- j. 日志：SLF4J + logback
- k. 容器化平台：Docker

7. 挑战和解决方案：讨论在项目过程中遇到的挑战，以及你是如何应对这些挑战的。这展示了你的问题解决能力和适应性。（见专属问题部分，可以挑几个优化相关的问题进行描述）

8. 学习和成长：（分享你从这个项目中学到的东西，以及这些经验如何帮助你成长为一个更好的专业人士。）

- 掌握企业真实软件研发流程
- 具备系统架构能力

- 理解并学习项目核心技术
- 具备项目设计能力
- 掌握项目部署运维能力

4. 你从这个项目中学到了什么？

掌握企业真实软件开发流程

- 本次课程将按照**真实企业项目开发流程**进行授课，掌握企业项目从立项到落地的所有阶段。
- 熟悉企业项目开发版本计划
- 理解并掌握分布式项目构成
- 理解并掌握企业项目环境隔离体系
- 理解并掌握企业项目代码管理手段

具备系统架构能力

- 掌握微服务系统架构能力
- 掌握微服务技术架构能力

理解并学习项目核心技术

- 项目集成了 Nginx、Nacos、MySQL、Vue3、微信小程序开放平台等 20 多个主流后端与前端组件，技术栈丰富且前沿。完成本次项目的学习，这些企业级开发常用的核心技术组件，都可以轻松掌握。

序号	分类	技术组件	组件说明
1	后端	bite-framework-java	【比特JAVA脚手架】为项目搭建提供基础支持。
2	后端	Spring Boot	快速构建独立运行的Spring应用，简化配置和部署。
3	后端	Spring Cloud	提供微服务架构的分布式解决方案（服务发现、配置管理等）。
4	后端	Redis	高性能内存数据库，用于缓存、会话存储和分布式锁。
5	后端	RabbitMQ	消息队列中间件，实现服务间异步通信和解耦。
6	后端	Nacos	动态服务注册发现和配置管理中心，支持服务治理。
7	后端	Mysql	开源关系型数据库，存储核心业务结构化数据。

8	后端	Mybatis/MybatisPlus	Java持久层框架，简化数据库操作（MybatisPlus增强CRUD功能）。
9	后端	Redisson	Redis分布式工具库，提供分布式锁、对象存储等高级功能。
10	后端	Jwt	跨域认证方案，用于安全传输用户身份信息。
11	后端	Nginx	反向代理服务器，处理负载均衡、静态资源托管和请求转发。
12	后端	WebSocket	双向实时通信协议，支持即时消息、在线咨询等功能。
13	运维	Docker	容器化平台，实现应用隔离和跨环境一致性部署。
14	运维	Docker Compose	多容器编排工具，通过YAML文件定义和管理容器集群。
15	三方对接	阿里云OSS	云对象存储服务，提供海量文件的安全存储与访问。
16	三方对接	APIFox	API协作平台，支持接口文档、测试和Mock数据生成。
17	三方对接	微信登录	集成微信认证，实现用户免密登录。
18	三方对接	腾讯地图	提供地理位置、POI 搜索等功能。
19	三方对接	阿里云短信	短信API服务，支持验证码消息发送。
20	三方对接	阿里云容器镜像仓库	托管Docker镜像的私有仓库，支持镜像版本管理。
21	三方对接	Gitee	国内代码托管平台，用于版本控制和团队协作。
22	前端-Web端	Vue3	渐进式JavaScript框架，构建响应式用户界面。
23	前端-Web端	Vue-Router	Vue官方路由管理器，实现单页面应用（SPA）导航。
24	前端-Web端	Pinia	轻量级状态管理库，替代Vuex管理全局数据流。
25	前端-Web端	Element Plus	基于Vue3的UI组件库，提供丰富的前端交互组件。
26	前端-Web端	Axios	Promise-based HTTP客户端，处理前端API请求。
27	前端-Web端	Vite	前端构建工具，支持极速开发热更新。
28	前端-小程序端	微信小程序开放平台	提供小程序开发、发布和运营的全套能力与API支持。

具备项目设计能力

- 理解并掌握数据模型设计的合理思路
- 理解并掌握高并发场景下的设计思路
- 设计并实现高并发场景下房源 Redis 缓存方案
- 理解并掌握分布式锁的使用场景
- 理解并实现单例、工厂、策略等设计模式在项目中的应用
- 理解并掌握高并发场景下异步编排的处理方式
- 理解并掌握高并发场景下实时通信技术的设计思路
- 理解并掌握 WebSocket 的应用场景与实现方式
- 设计并实现咨询消息的核心处理流程
- 设计并实现基于 WebSocket 与 RabbitMQ 的异步高性能消息处理方案
- 设计并实现咨询消息的异步持久化方案
- 设计并实现高并发场景下，咨询消息的 Redis 缓存方案

掌握项目部署运维能力

- 掌握 Docker 实现应用隔离和跨环境一致性部署。
- 掌握企业级实际部署方案，如插件一键式部署、流水线部署方式。

5. 你在这个项目中使用了哪些技术？为什么选择(决策)这些技术？

- 关联问题：
 - 为什么缓存选择 Redis?
 - 为什么消息队列选择 RabbitMQ?
 -

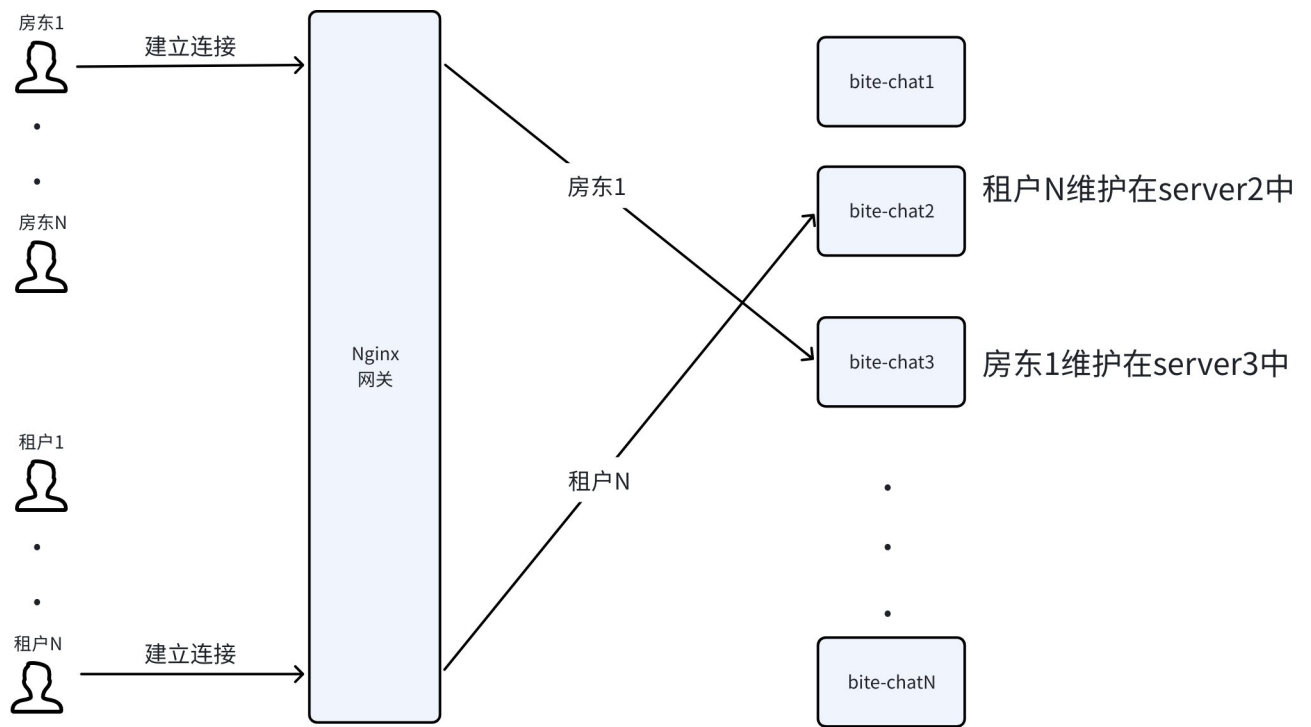
序号	分类	技术组件	组件说明
1	后端	bite-framework-java	【比特JAVA脚手架】为项目搭建提供基础支持。
2	后端	Spring Boot	快速构建独立运行的Spring应用，简化配置和部署。
3	后端	Spring Cloud	提供微服务架构的分布式解决方案（服务发现、配置管理等）。

4	后端	Redis	高性能内存数据库，用于缓存、会话存储和分布式锁。
5	后端	RabbitMQ	消息队列中间件，实现服务间异步通信和解耦。
6	后端	Nacos	动态服务注册发现和配置管理中心，支持服务治理。
7	后端	Mysql	开源关系型数据库，存储核心业务结构化数据。
8	后端	Mybatis/MybatisPlus	Java持久层框架，简化数据库操作（MybatisPlus增强CRUD功能）。
9	后端	Redisson	Redis分布式工具库，提供分布式锁、对象存储等高级功能。
10	后端	Jwt	跨域认证方案，用于安全传输用户身份信息。
11	后端	Nginx	反向代理服务器，处理负载均衡、静态资源托管和请求转发。
12	后端	WebSocket	双向实时通信协议，支持即时消息、在线咨询等功能。
13	运维	Docker	容器化平台，实现应用隔离和跨环境一致性部署。
14	运维	Docker Compose	多容器编排工具，通过YAML文件定义和管理容器集群。
15	三方对接	阿里云OSS	云对象存储服务，提供海量文件的安全存储与访问。
16	三方对接	APIFox	API协作平台，支持接口文档、测试和Mock数据生成。
17	三方对接	微信登录	集成微信认证，实现用户免密登录。
18	三方对接	腾讯地图	提供地理位置、POI 搜索等功能。
19	三方对接	阿里云短信	短信API服务，支持验证码消息发送。
20	三方对接	阿里云容器镜像仓库	托管Docker镜像的私有仓库，支持镜像版本管理。
21	三方对接	Gitee	国内代码托管平台，用于版本控制和团队协作。
22	前端-Web端	Vue3	渐进式JavaScript框架，构建响应式用户界面。
23	前端-Web端	Vue-Router	Vue官方路由管理器，实现单页面应用（SPA）导航。

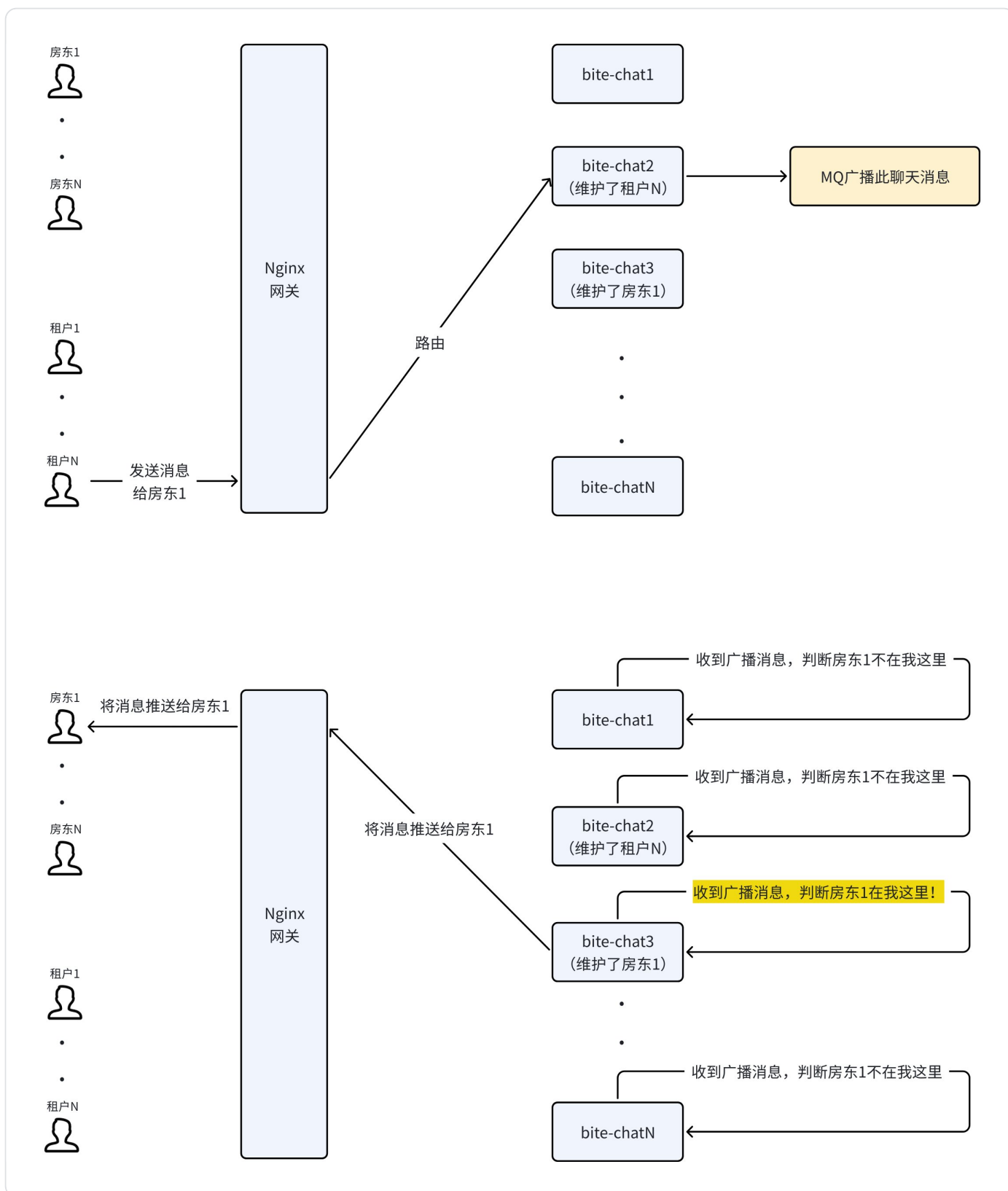
24	前端-Web端	Pinia	轻量级状态管理库，替代Vuex管理全局数据流。
25	前端-Web端	Element Plus	基于Vue3的UI组件库，提供丰富的前端交互组件。
26	前端-Web端	Axios	Promise-based HTTP客户端，处理前端API请求。
27	前端-Web端	Vite	前端构建工具，支持极速开发热更新。
28	前端-小程序端	微信小程序开放平台	提供小程序开发、发布和运营的全套能力与API支持。

- **数据库：MySQL + MyBatis**
 - MySQL 和 MyBatis 是数据存储和访问的一个常见的做法。MySQL 是一个开源的关系数据库管理系统，它可以免费使用。MyBatis 提供了一种相对简单的方法来执行 SQL 语句，不需要编写复杂的 ORM 映射文件。
- **缓存：Redis**
 - 我需要将热点数据存入缓存中，因为这些内容是用户常常访问的，虽然我只是实现了项目，并没有很多人使用，但我判断如果是真实上线的项目，这部分流量是较大的。为了提高系统性能，我选择了 Redis，它是基于内存的存储系统，其读写速度非常快，可以显著提升数据访问速度。
 - 延申问题：redis的其他特点 (推荐学习redis精品课程):
 - **高性能**：是基于内存的存储系统，其读写速度非常快，可以显著提升数据访问速度。
 - **丰富的数据结构**：Redis支持多种数据结构，如字符串、哈希、列表、集合、有序集合等，这使得它能够适应各种不同的缓存需求。
 - **持久化支持**：虽然Redis是内存数据库，但它提供了数据持久化的选项，可以将内存中的数据保存到磁盘上，确保数据的安全性和可靠性。
 - **高并发处理能力**：Redis能够处理大量的并发请求，这对于高流量的Web应用来说非常重要。它能够减少对后端数据库的直接访问，从而降低数据库的负担。
 - **分布式缓存**：Redis支持主从复制和哨兵模式，可以构建高可用性的集群架构，提供数据的备份和故障转移功能，保证系统的稳定运行。
 - **解决缓存问题的能力**：Redis能够有效应对缓存穿透、缓存雪崩和缓存击穿等问题，通过设置合理的过期时间、使用布隆过滤器等策略来保证系统的稳定性
- **消息队列：RabbitMQ**

集群环境下，每台服务器只维护了和本服务器内连接的session 信息。若出现下图情况，房东1与 租户N 并不能正常进行咨询，因为 server2 无法向 房东1 发消息，server3 也无法向 租户N 发消息。如何解决？



借助 RabbitMQ 消息队列将咨询消息进行**广播**处理。



到这里我们已经看到了一个消息可以正常的进行发送与接收，但不要忘记，对于用户发送的消息来说，服务器是要将其持久化存储下来的，以便以后可以查询历史咨询记录。

◦ 延申问题：RabbitMQ 的特点 (推荐学习RabbitMQ精品课程)：

- **异步处理能力**：RabbitMQ 能够将耗时的任务异步处理，提高系统的响应速度和吞吐量，使得系统能够更快地响应用户请求。
- **解耦和灵活性**：通过使用RabbitMQ，生产者 and 消费者之间的耦合度降低，它们不需要直接交互，从而提高了系统的灵活性和可维护性。

- **可靠性和持久性：**RabbitMQ支持消息的持久化和可靠传输，确保消息不会在传递过程中丢失，这对于需要高可靠性的应用程序至关重要。
- **路由和消息筛选：**RabbitMQ提供了灵活的路由和消息筛选机制，支持多种交换机类型，可以根据消息的特性将消息路由到不同的队列中。
- **社区支持和成熟性：**作为一个开源项目，RabbitMQ拥有活跃的社区支持和广泛的用户范围，具有较高的稳定性、可靠性和成熟性。
- **消息确认和死信队列：**RabbitMQ提供了消息确认机制和死信队列，确保消息在消费过程中的可靠性，以及在消费失败时能够有相应的处理策略。

● **日志：**SLF4J + logback

- SpringBoot 选用 SLF4j和logback，我参照了它。

● **身份验证：**JWT

- 传统的验证方式：登陆页面把用户名密码提交给服务器；服务器端验证用户名密码是否正确,并返回校验结果给后端；如果密码正确,则在服务器端创建 Session（或 Token）,并保存，通过 Cookie 把 sessionId（或 Token）返回给浏览器，下次请求时，带上对应内容，后端通过查库进行校验。
- JWT一种行业都认可的开发标准，是用于web开发的JSON形式的token，我们现在的开发模式都是前后端分离，数据传输就是用的JSON。且这个token有标准，有定义规范。也导致了使用时不用保存到redis数据库中，而是可以直接解析进行验证。

6. 你在项目中扮演了什么角色？你负责了哪些模块或功能？

本项目是一个 Java 综合项目，本项目是一个 Java 综合项目，涵盖【前端】与【后端】开发，我负责后端部分与部署的所有内容，包括需求的设计，功能的拆分，后端的开发与部署等。前端开发由我同学负责。在这个过程中，也是我来对项目节奏进行把控。

7. 在项目中，你是如何平衡技术实现和项目时间线的？这个项目做了多长时间？

这个项目我花了2个多月来完成，从项目立项，到需求设计，到人员分配，到需求评审、再到最后的开发、测试、部署。

关于时间，我们制定详细的规划。

立项	● 背景调研 ● 收集需求 ● 分析需求 ● 立项
----	---

需求设计	• 编写需求文档（PRD） • UI 展示
人员分配	• Gitee 项目创建 • 人员分配 • 技术负责人通知对应服务负责人
需求评审	• 组织需求评审会议 • 与开发、测试人员进行需求评审
方案与接口评审	• 接口定义与方案产出 • 组织后端方案与接口评审会议 • 与技术负责人、测试人员进行评审
项目开发	• 后端开发 ◦ 需求分析 ◦ 系统方案设计 ◦ 开发环境搭建 ◦ 编写代码、自测 ◦ 代码提交（Git） ◦ 部署联调&测试环境 • 前端开发 ◦ 前端开发 ◦ 项目联调
项目测试	• 准备测试用例 • 进行测试 • Bug提单 • 修改Bug • Bug回归 • 关单 • 回归测试
部署上线	• 生产环境搭建 • develop合并master • 流水线部署 • 线上验证

回复思路可参考：

1. **明确项目范围和目标：**在项目开始之前，我确定了项目到底要做什么，对范围、目标和期望有清晰的认知。
2. **制定详细的计划（如果有）：**创建一个详细的项目计划，包括项目原型图的敲定，到前后端功能的拆分，前后端交互接口和各功能的联调截至时间。
3. **优先级排序：**确定哪些功能和特性是当下最重要的，并根据优先级进行排序。这有助于在时间有限的情况下做出决策。比如支持抽奖行为，就必须先完成管理端。
4. **人员分配（如果有）：**由于前端是交付给其他同学完成，以上内容也需要和前端同学对齐。（如果说前端也是自己完成的，就不用说这一点）
5. **沟通和协作（如果有）：**确保和前端同学有良好的沟通和协作，以便快速解决问题并共享信息。（如果说前端也是自己完成的，就不用说这一点）
6. **质量保证：**计划中还包含足够的时间来测试项目，进行质量保证。

8. 你在项目中遇到了哪些挑战？你是如何识别和解决这些问题的？

非技术方面：

- 由于没有产品经理角色的介入，因此对于该项目的设计需要我自己把控，需要画原型草稿图、功能的拆分等等对我来说都是一个挑战，但我认为不懂业务的java开发不是好开发，因此我也愿意去了解需求，多了解一些业务到底要怎么设计。

技术方面：

- （见专属问题部分，可以挑几个优化相关的问题进行描述）

9. 你是如何测试和部署项目的？

测试：

1. **单元测试：**编写测试代码来验证每个独立模块或函数的正确性。
2. **集成与功能测试：**在单元测试之后，测试各个模块如何一起工作，以及功能接口是否正确返回。使用APIFox进行接口测试。
3. **日志打印：**使用日志打印来记录、分类和修复发现的问题。

部署：

- 生产环境搭建
- develop合并master
- 流水线部署
- 线上验证

10. 你是如何优化项目性能的？你有哪些性能调优的经验？

- 房源搜索的筛选+排序用设计模式优化
- 热点数据存放缓存的优化
- 没有性能调优的经验。

11. 你是如何确保项目安全性的？你使用了哪些安全措施？

- 见脚手架专属问题

12. 你是如何设计项目以支持未来的扩展？你是如何确保代码的可维护性的？

设计模式的使用：硬编码筛选/排序逻辑导致代码臃肿。

解决方案：

- 策略模式：定义 `IHouseFilter`（区域/租金筛选）和 `ISortStrategy`（价格/距离排序）；
- 工厂模式： `SortStrategyFactory` 根据参数动态选择排序策略；
- 优势：新增筛选条件只需实现新类，无需修改主逻辑。

代码块

```
1 // 策略模式示例
2 List<HouseDTO> filtered = houseList.stream()
3     .filter(house -> filterMap.values().stream()
4         .allMatch(filter -> filter.apply(house, reqDTO)) // 组合多个筛选器
5     .collect(Collectors.toList());
```

咨询消息的持久化优化

分库分表：按 `user_id` 哈希分库，按月份分表（如 `chat_msg_2023_10`）；

读写分离：

- 写操作：主库（高并发下队列缓冲）；
- 读操作：从库 + Redis缓存最近10条消息；

冷热分离：超过3个月的消息转存至Elasticsearch。

房源详情页QPS达到1万的优化

分层优化：



具体措施：

- Redis 缓存命中率提升：预热热点房源；
- MySQL：添加读写分离 + 二级缓存（Caffeine）；
- 限流：Sentinel对高频查询 `/detail` 接口限流。

脚手架专属问题

1. 脚手架项目的意义是什么？（设计思想、目的）

从项目开发角度看，每次启动新项目都要做许多重复工作：引入网关、服务注册与发现中心等基础组件，封装统一异常处理、统一响应结构，构建用户登录注册逻辑，此外还有大量基础通用配置。这些工作在不同项目中几乎一致。而脚手架项目封装了这些重复通用内容，让后续开发无需从零开始——既能减轻开发者工作量，也能帮助其基于脚手架快速启动新项目。

从个人学习角度来看，从零搭建脚手架项目，让我清晰掌握了项目的开发流程与设计思想，也帮助复习并深化了对所学知识的理解，实现知识融会贯通。对于登录注册等通用业务理解也更深刻，未来能快速集成到新项目中；这也利于我更快适应企业开发的模式。

2. 脚手架项目和租房项目的关系是什么？

脚手架项目和租房项目关系如下：

- 租房项目基于比特Java脚手架项目进行开发。脚手架项目和租房项目之间的关系是“基础框架”与“具体业务实现”的关系。它们共同构成完整的应用。
 - 脚手架项目提供基础设施，如：Nacos集成方案、网关集成方案、基础通用工具类、统一状态码、统一数据结构等。
 - 租房项目模块实现租房相关业务，如：房源列表、房源详情、咨询业务等。
- 脚手架项目可以独立提供给其它项目
 - 脚手架项目作为项目的基础框架，不仅租房项目可以基于其开发，未来开发新项目都可以基于脚手架项目进行开发。

3. 脚手架项目中封装统一响应数据结构的意义是什么？

注意：以统一响应数据结构为例，脚手架项目中所有封装的模块和组件均可套用此回答。

脚手架项目中所有封装（组件/模块）共同意义：

- 消除“从零造轮”的启动成本：通过封装在脚手架中的通用代码和配置信息，减少开发者的重复性工作，提高开发效率。
- 统一开发规范：统一开发规范，如代码风格、命名规范等，降低沟通成本，提高代码质量。

封装统一响应数据结构的核心意义：

- 统一了后端接口的开发规范，统一接口返回值，降低前后端沟通成本，减少前端开发成本。提高开发效率。
- 统一响应数据结构包含统一状态码、消息、返回数据，以及多个用于构建成功 / 失败返回值对象的重载方法。该结构可快速封装统一格式的响应数据，既便于前端高效处理，若出现问题也能依据返回信息快速定位排查。

4. 统一异常处理是什么？是怎样进行处理的？

- 统一异常处理：是一种架构设计模式，核心是通过集中化方式管理并响应系统运行中的各类异常。简单讲，接口发生异常时，若不处理，就无法向使用者返回准确提示；但如果每个接口自行处理，既会导致代码冗余、维护困难，还会出现异常响应不统一的问题。因此，统一异常处理旨在解决传统方式的这些痛点，进而提升系统的健壮性、可维护性与用户体验。
- 在比特Java脚手架项目中异常处理分为两部分分别是：（1）网关层异常 （2）业务层异常
 - 对于网关层异常：需要在网关服务层实现ErrorWebExceptionHandler接口，当异常发生时，实现ErrorWebExceptionHandler接口的类的handle方法会自动调用对异常进行统一处理。并返回全局通用的请求响应数据结构。
 - 对于业务层发生的：通过 `RestControllerAdvice+ExceptionHandler` 进行处理，返回全局通用的请求响应数据结构。

注意：可参考课件《比特Java脚手架-后端上》统一异常处理部分。

5. Java已经有内置的异常类为什么还需要自定义异常？

- 增加代码的可读性同时也符合业务规则和语义：业务逻辑往往有其自身的规则和限制，自定义异常能够更好地体现这些规则。使用自定义异常能够使得代码更加易于理解和维护。
- 满足业务需求：Java提供了一些内置的异常类，但是它们并不一定能够满足我们的具体业务需求，所以我们需要自定义异常类。

6. 为什么需要Bean拷贝工具类，项目中为什么要用多种数据模型？

- 为什么需要Bean拷贝工具类：
 - 在Java web项目中通常将项目分为：控制器（Controller）层、业务逻辑层（service）层、数据访问（DAO）层。而在不同层之中我们通常会使用不同的数据模型。这也就意味着项目中往往不止一种数据模型，我经常需要将数据从一种数据模型转换为另外一种数据模型，正是因为如此才需要Bean拷贝。
- 为什么要划分不同的数据模型：

- **提高代码的可读性和可维护性：**每种对象都有其特定的用途和职责，使得代码结构更加清晰，开发人员能够更容易地理解数据的流动和使用方式。每个对象采取统一的命名，项目会变得更加一致和可预测，从而减少了出错的可能性。
- **解耦：**DO、BO、DTO、VO等分别作用于不同层，他们的划分降低了各个部分的耦合度。修改某一层的逻辑或数据时减少对于其它层的影响。
- **优化性能：**不同的对象，比如DTO和VO可以根据自身的功能和当前的需求，进行裁剪和拼装。合理的利用网络传输资源，提高性能。

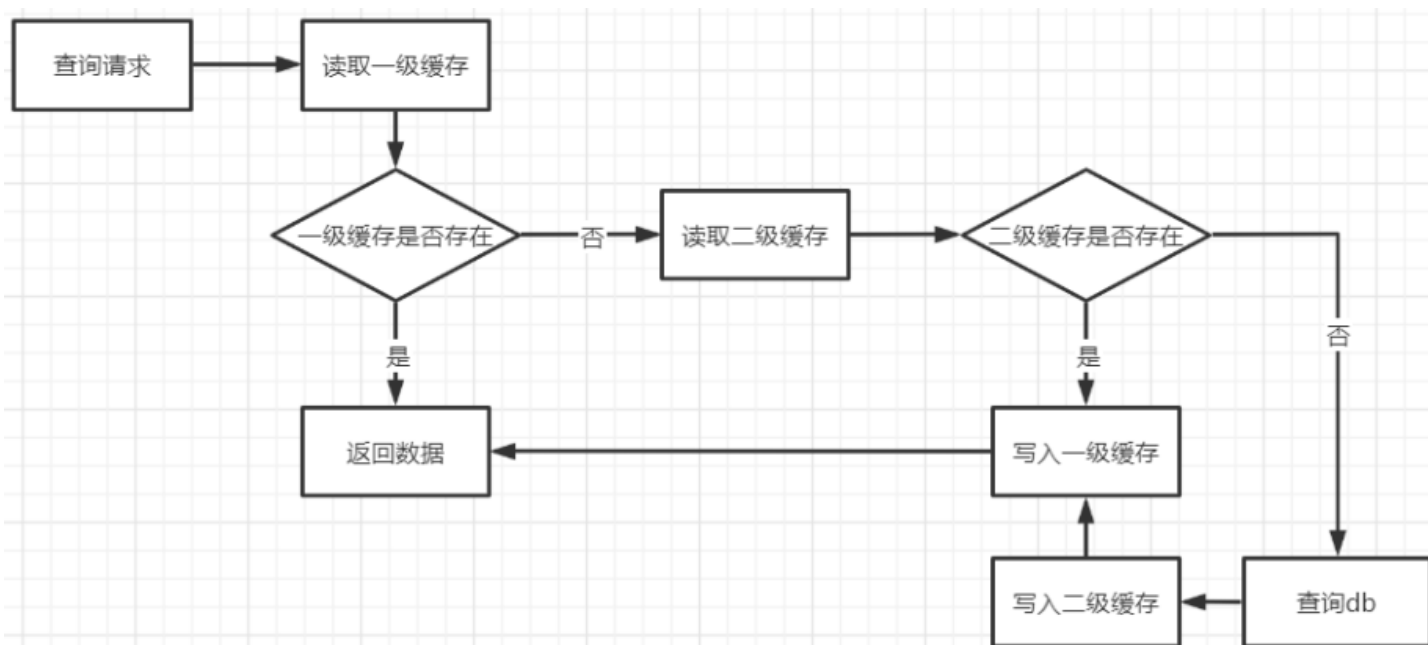
7. 你如何实现的二级缓存？通过二级缓存能够解决什么问题？

在脚手架项目当中我引入了Caffeine作为本地缓存并将本地缓存作为一级缓存，引入Redis作为二级缓存，二者与MySQL搭配，组成两级缓存。

当系统收到查询请求后，先进入缓存查询阶段。

- 缓存查询阶段：
 - 首先尝试读取一级缓存，若一级缓存存在数据，直接将数据返回给调用方。若一级缓存不存在数据，进入二级缓存查询阶段。
 - 如果二级缓存存在数据先将数据写入一级缓存，再返回数据。若二级缓存也不存在数据：进入数据库查询阶段。
- 数据库查询阶段：
 - 一二级缓存均未命中，被迫查询数据库（DB）获取原始数据，随后将数据分别写入二级缓存和一级缓存，最后返回数据。

注意：记忆时可通过图形记忆：



二级缓存能够解决的问题：

- 大幅提升系统性能：缓存的加入使得系统响应速度更快，整体性能得到显著提升。

- b. 加速热点数据访问：一级缓存直接存储极高频访问的数据，抗住最热请求。
- c. 扩展缓存容量上限：二级缓存存储更大数据集。
- d. 优化内存资源利用率：高频小数据存一级缓存，低频大数据存二级缓存，降低Redis内存占用成本
- e. 有效减轻数据库压力：缓存机制降低了数据库的访问频率，减少了数据库的负载
- f. 应对缓存失效风暴：通过本地缓存分散Redis失效时的请求峰值，结合互斥锁/空值缓存预防雪崩与击穿。

8. 请介绍下你的文件服务架构

文件服务一共分为三层分别是：

- 用户交互层：前端。
- 核心中枢层：就是我所开发的文件微服务。
- 文件存储层：我所采用的是阿里云OSS对象存储。

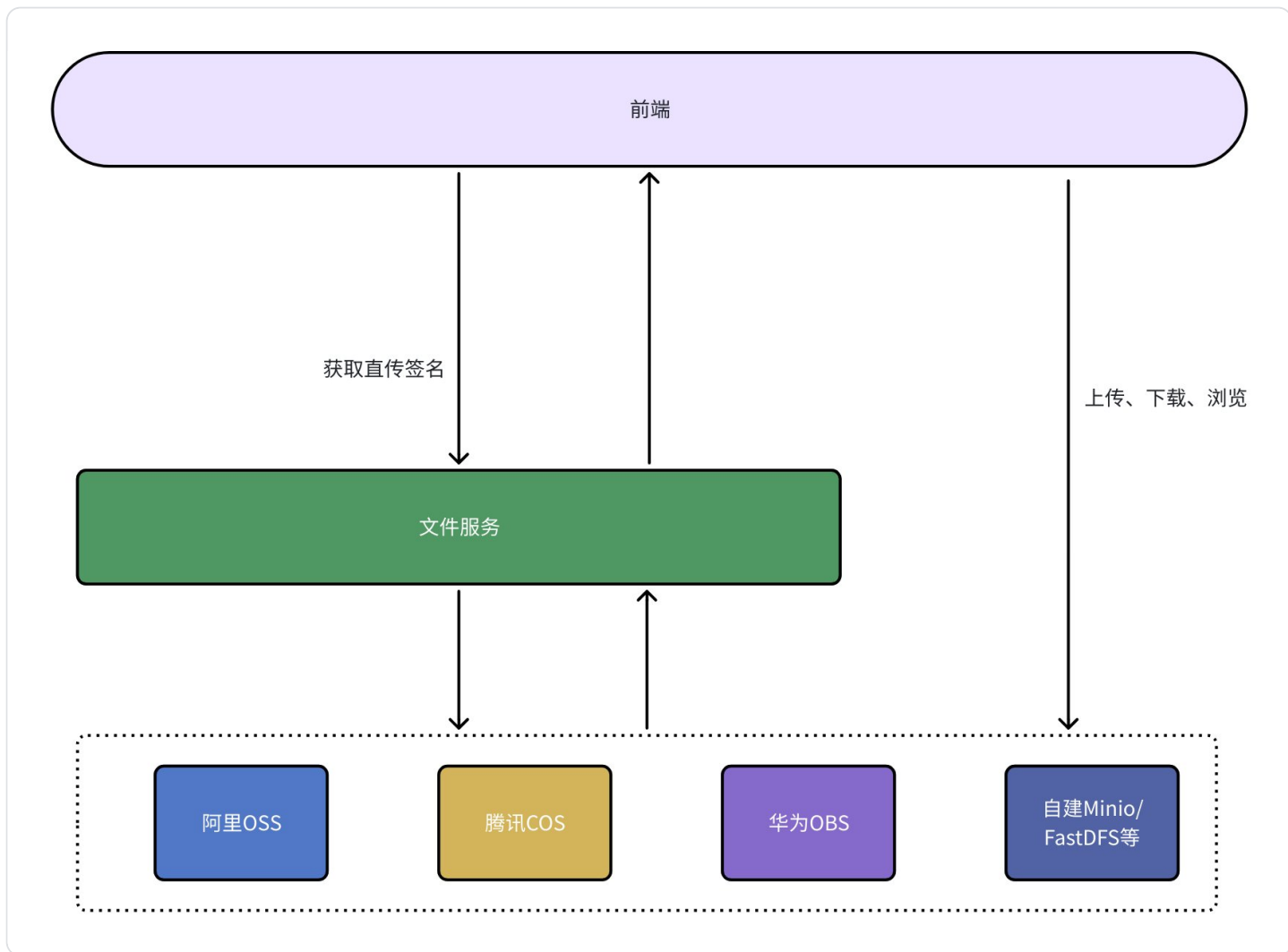
当上传文件时：

采用前端直传方案，当用户触发上传文件请求后，前端会先请求后端接口获取直传签名，然后再采取前端直传方式将文件上传至文件存储机制oss。

当下载文件时：

前端直接请求对象存储机制oss完成文件的下载、浏览。

架构图如下：



9. 为什么文件服务中引入了OSS对象存储文件不使用本地文件存储或其他存储机制如MySQL等？

引入oss对象存储机制的主要原因如下：

- 无限扩展性：
 - 本地：物理容量/性能有限，扩展需加硬件，复杂昂贵。
 - MySQL：分库分表复杂，存储海量文件效率极低，成本过高。
 - OSS：架构天然支持近乎无限的水平扩展，按需自动扩容。
- 高可用与持久性：
 - 本地：单点故障风险高，需复杂备份/容灾方案。
 - MySQL：文件存储易成单点，备份/恢复海量文件慢。
 - OSS：数据自动多副本/纠删码跨设备/区域存储，故障自愈，持久性极高。
- 成本效益：
 - 本地：前期硬件投入大，运维（电力/人力）成本高，资源易浪费。
 - MySQL：许可/计算资源成本高，存储文件性价比极低。

- OSS：按使用量付费，无前期投入；存储单价低；支持冷热分层进一步降本。
- 性能与并发：
 - 本地：单机I/O易成瓶颈，难应对高并发。
 - MySQL：文件I/O消耗核心数据库资源，性能差，影响业务。
 - OSS：分布式架构支持高并发访问；易集成CDN加速全球读取。
- 简化运维：
 - 本地/MySQL：需专业团队维护硬件/软件/备份/扩容，复杂耗时。
 - OSS：全托管服务，云商负责底层运维，用户通过API/控制台管理，负担轻。
- 丰富功能：
 - 本地/MySQL：实现高级功能困难且成本高。
 - OSS：开箱即用：生命周期管理、版本控制、细粒度权限、加密、图片处理、CDN集成、静态网站等。
- 聚焦核心：
 - OSS 让企业无需自建维护复杂存储设施，专注核心业务开发。

10. 请介绍下脚手架项目的分支策略？为什么要有分支策略？

为什么要有分支策略：回答时根据实际情况选择回答一或二，再参考多分支策略的具体好处补充说明。

- 回答一（如果前面说项目后端是一个人开发）：据我了解企业中项目都是合作开发，并且采用多分支策略实现高效的并行协作。为了快速适应企业开发模式熟练git操作，所以我采用多分支策略模拟多人协作模式。
- 回答二（如果前面项目回答了后端是多人开发）：采用多分支策略可以实现高效的并行协作，让我们团队的工作推进更加顺利，成员之间工作不互相影响。
- 多分支策略的具体好处：
 - 并行开发与隔离：不同的开发者可使用不同的分支进行开发，避免相互干扰。
 - 环境稳定性保障：不同的环境（开发/测试/生成.....）都对应特定的分支确保环境稳定可靠。
 - 灵活发布控制：我的分支策略中包含bug分支，当线上出现问题时可灵活拆分bug分支修改并及时发布。

具体分支策略如下：

- **master 分支：**用于部署到正式发布环境。此分支为唯一分支，并且会被设置为保护分支（不允许任何人推送，只允许仓库管理员合并）。

- **develop 分支**：含义为开发分支，基于 master 分支创建的唯一分支，并且会被设置为保护分支，始终始终保持最新完成（自测通过）以及 bug 修复后的代码（测试环节发现的bug）。可部署到联调或测试环境。
 - 如果master分支有变更需要及时同步。
- **feature 分支**：通常为新功能或新特性开发分支，以 `develop` 分支为基础而创建出的分支。
 - 命名以 `feature/` 开头，后面拼接功能名称。例：`feature/map`，多个单词之间可以用短线分割。
 - 新特性或新功能开发完成后，开发人员需合到develop分支，合入develop分支后将其删除。
 - 如果develop分支有变更需要及时同步。
- **bug分支**：修改bug的分支，线上bug基于master分支创建出分支，测试过程发现的bug基于develop分支创建出分支。
 - 命名以 `bug/` 开头，后面拼接bug名称（能指明是什么bug），多个单词之间可以用短线分割。
 - 如果是线上bug，直接将bug分支部署到测试环境进行测试，测试通过后合入master分支重新发布上线，上线成功后删除bug分支。
 - 非线上bug修复完成后合入develop分支，合入后重新将develop分支代码部署到测试环境，测试人员对所修改bug验证无误后可删除bug分支。

11. 你在接入腾讯地图服务的时候都遇到了什么问题？

在接入腾讯地图服务时，主要遇到以下问题：

1. API调用限制：免费版有每日调用限额，突发流量可能导致服务中断，需升级付费版本或优化调用频率。
2. 定位偏差：尤其在复杂建筑或偏远地区，GPS定位可能出现偏移，需结合Wi-Fi或基站数据辅助校准。
3. 文档不清晰：部分接口文档描述模糊，例如逆地理编码的参数格式，需反复测试或联系技术支持。

12. 讲一下你对字典服务和参数服务的理解？为什么要这么拆分？

字典服务通常用于管理系统中的静态数据，如国家/地区列表、性别、订单状态等。这些数据具有以下特点：

- 变更频率低：如“性别”一般不会频繁修改。
- 全局通用性：多个业务模块可能共用同一套字典数据（如“订单状态”可能被订单、物流等多个服务引用）。
- 数据量较小：通常以KV（Key-Value）结构存储，便于快速查询。

参数服务用于管理动态配置，如系统开关、业务规则阈值、超时时间等，特点包括：

- 变更频率高：例如促销活动的折扣率可能随时调整。

- 业务相关性：不同模块可能有独立参数（如支付超时时间、风控规则）。
- 支持动态生效：通常结合配置中心实现实时更新，避免重启服务。

拆分依据如下：

- 解耦与复用：字典数据通常全局共享，独立服务避免重复存储；参数服务则按业务划分，减少耦合。
- 维护清晰：静态字典和动态配置的变更管理策略不同，拆分后更易于维护。

13. 为什么要设计远程调用和HTTP接口调用两种调用方式？

远程调用性能更高，适合服务间高频调用。HTTP兼容性强，跨系统，跨语言适合前后端分离的场景。

14. 你把用户服务模块拆分成B端用户和C端用户分的依据是什么？

把用户服务拆分成B端用户与C端用户的依据如下：

B端用户通常是企业内部员工或者商业用户，这些用户的特点是用户量小，权限高，注册途径少（超级管理员开通）通常是先开通，再登录，对安全要求较高。

C端用户通常是普通消费者用户，这些用户的特点是用户量大，权限低，注册途径多（微信一键登录、短信验证码登录、QQ一键登录）通常是注册即登录尽可能要减少用户的操作次数，降低用户操作成本。

拆分B端用户和C端用户分的本质是业务差异驱动拆分，避免“一刀切”导致系统臃肿或性能瓶颈。

15. 短信登录服务，手机验证码为什么要设置有效期？

手机验证码设置有效期是出于安全性、用户体验和系统性能的综合考虑，主要原因包括：

- 防止暴力破解与重放攻击
 - 暴力破解：如果验证码长期有效，攻击者可反复尝试（如遍历6位数字组合），增加破解概率。
 - 重放攻击（Replay Attack）：攻击者截获旧验证码后重复使用（如通过木马窃取短信），有效期限制可降低此类风险。
- 提升用户体验
 - 时效性：用户通常在短时间内完成操作（如登录/注册），过期后需重新获取，避免因旧验证码混淆导致流程卡顿。
 - 减少误操作：例如用户输错手机号，有效期可自动失效，避免无效验证码被滥用。
- 优化系统资源
 - 存储压力：服务端无需长期保存验证码记录，减少Redis等缓存的存储负担。
 - 清理机制：自动过期可简化垃圾数据清理逻辑（如定时任务）。

16. 为什么你脚手架项目的部署方案要区分单机版和集群版？

在设计脚手架项目的部署方案时，区分单机版和集群版主要基于以下考虑：

- 单机版：适用于开发测试、小型项目或个人学习，资源需求低（如1核2G服务器）。快速启动，无需复杂依赖。
- 集群版：面向生产环境、高并发或企业级应用，需要高可用和横向扩展能力。依赖分布式组件部署。

提供两种方案有以下好处：

- 降低门槛：单机版让用户快速体验核心功能，避免初期陷入复杂部署。
- 灵活演进：用户可从单机版过渡到集群版，逐步适应生产需求。

业务专属问题

1. 为什么将房源表拆分为水平分表（如按状态分表）？ 具体如何实现？

- 原因：
 - 单表数据量过大（如房源状态频繁更新）会导致查询性能下降；
 - 不同状态（上架/已租/下架）的查询频率不同，拆分可减少锁争用；
 - 便于维护（如清理历史数据时不影响活跃数据）。
- 实现：
 - 通过 `house_status` 表独立存储状态字段，业务层通过 `house_id` 关联查询。更新状态时只需操作小表，核心SQL示例：

代码块

```
1  -- 查询已出租房源
2  SELECT h.* FROM house h
3  JOIN house_status s ON h.id = s.house_id
4  WHERE s.status = 'RENTING';
```

2. 描述房源缓存的“双写一致性”方案，如何解决缓存穿透？

- 双写一致性：采用 Cache-Aside 模式：
 - 写 DB 后同步更新 Redis（加分布式锁保证原子性）；
 - 读请求先查缓存，未命中则查DB并回填。
- 缓存穿透解决：
 - 空值缓存：查询DB不存在时，在Redis缓存 `NULL`（设置 TTL）；

- 在Redis层预过滤非法 `house_id`。

3. 为什么用CompletableFuture获取城市列表？对比传统串行调用的优势？

- 优势：
 - 并行化：`allCityMap`（全量城市）和 `hotCityList`（热门城市）的Feign调用并行执行；
 - 降低延迟：若单个 Feign 调用耗时100ms，串行需200ms，并行仅需100ms。
- 代码关键点：

代码块

```
1 CompletableFuture<List> hotFuture = CompletableFuture.supplyAsync(() ->
  getHotCity());
2 CompletableFuture<Map> allFuture = CompletableFuture.supplyAsync(() ->
  getAllCity());
3 CompletableFuture.allOf(hotFuture, allFuture).join(); // 阻塞等待所有完成
```

4. 定时任务如何解决“房源状态扭转”的分布式并发问题？

即多实例同时执行任务会导致房源状态重复更新。

解决方案：

- Redisson分布式锁：对任务加锁（Key= `scheduledTask:lock`）；
- 唯一性：仅获取锁的实例执行DB查询和状态更新。

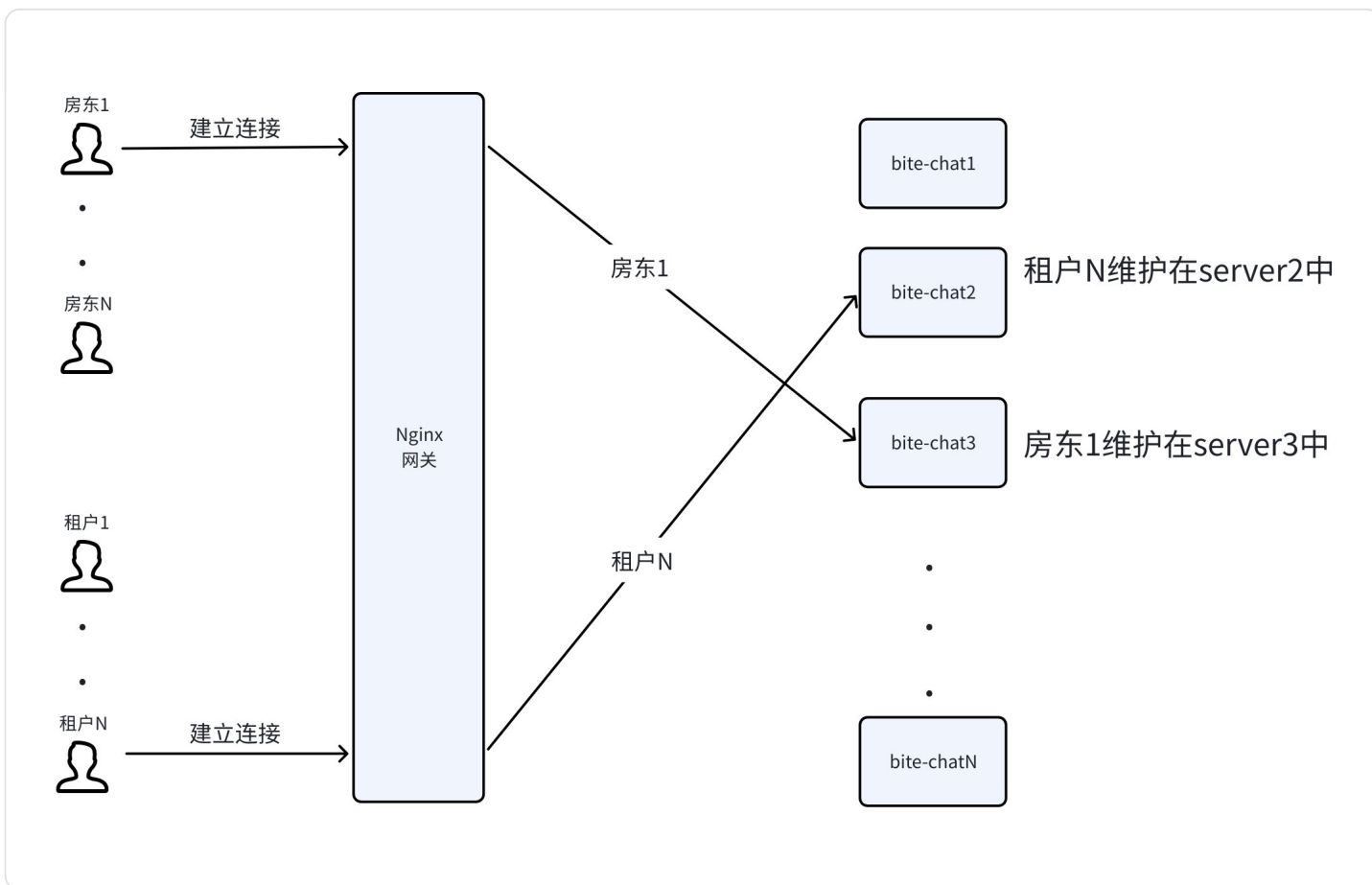
5. WebSocket相比HTTP轮询在消息模块的优势是什么？

维度	WebSocket	HTTP轮询
连接开销	1次TCP长连接	频繁建立/断开TCP
实时性	毫秒级	依赖轮询间隔（如1s）
服务器压力	低（事件驱动）	高（QPS=用户数/间隔）
适用场景	咨询消息实时推送	低频更新场景

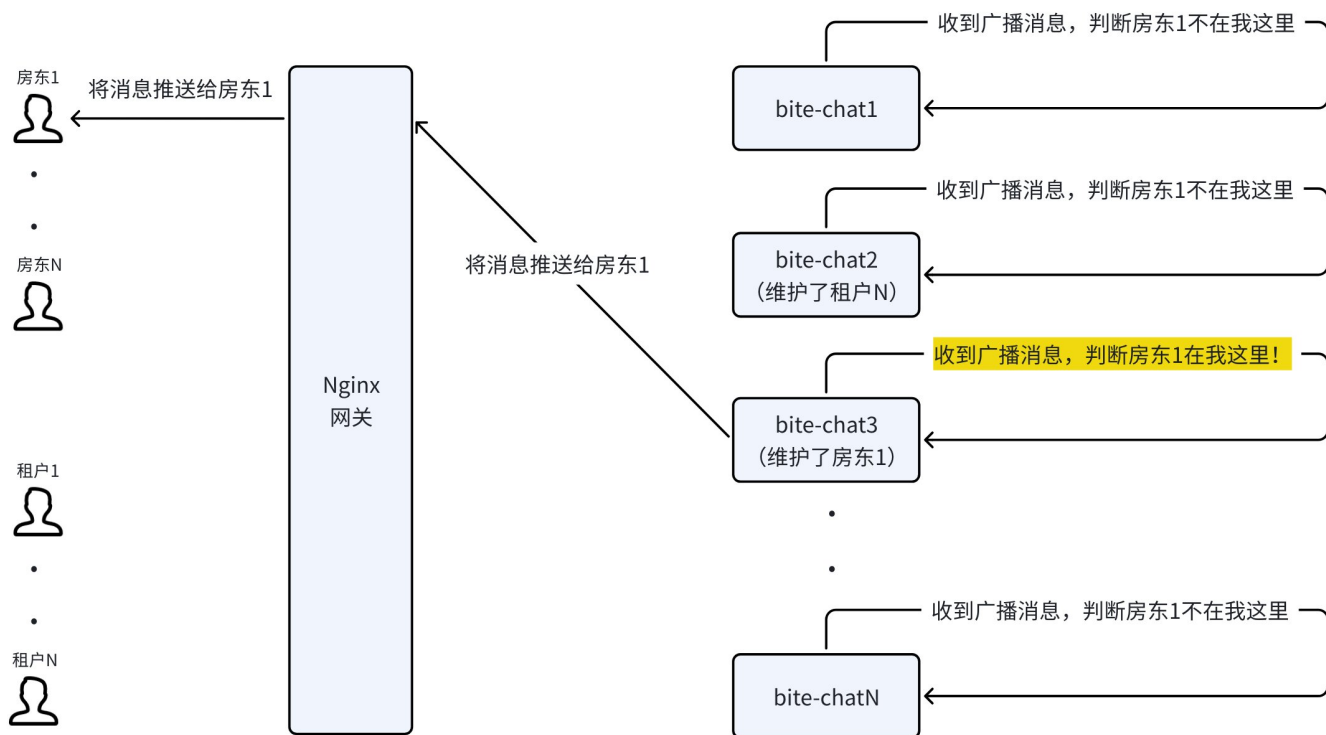
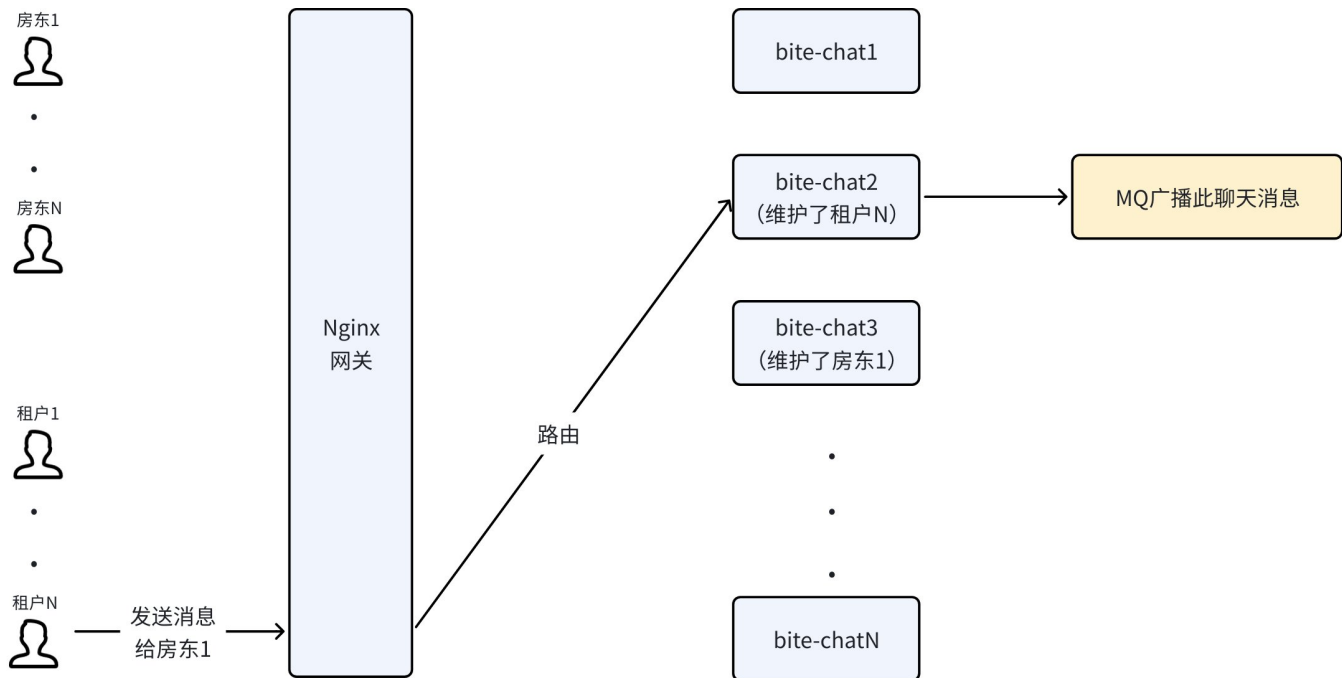
6. 咨询消息的“未浏览/已浏览”状态如何实现？

- 存储设计：消息表添加 `visited` 字段（10未浏览 11浏览过）。
- 更新逻辑：
 - 客户端行为：用户进入聊天页时，调用接口批量标记消息为已浏览；
 - 服务端推送：通过WebSocket主动通知对方消息已读状态。
- 优化：使用Redis Bitmap存储海量用户的未读计数，降低 DB 压力。

7. 集群环境下，每台服务器只维护了和本服务器内连接的session 信息。若出现下图情况，房东1 与 租户N 并不能正常进行咨询，因为 server2 无法向 房东1 发消息，server3 也无法向 租户N 发消息。如何解决？



借助 RabbitMQ 消息队列将咨询消息进行广播处理。

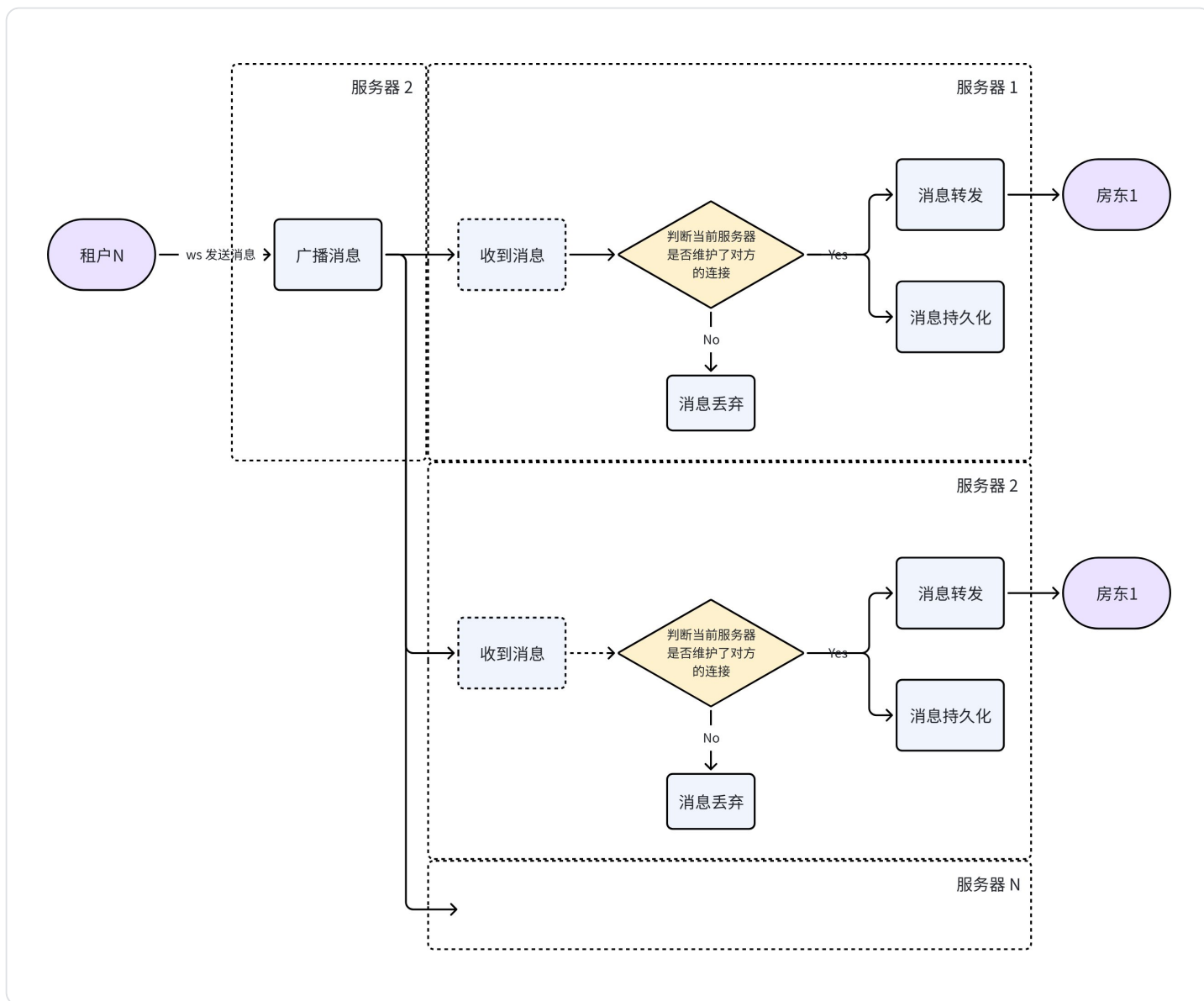


到这里我们已经看到了一个消息可以正常的进行发送与接收，但不要忘记，对于用户发送的消息来说，服务器是要将其持久化存储下来的，以便以后可以查询历史咨询记录。

因此对于接收到的消息处理，我们分为三个部分来完成：

1. 咨询消息的广播
2. 咨询消息的转发

3. 咨询消息的持久化



8. 房源搜索的筛选+排序如何用设计模式优化？

问题：硬编码筛选/排序逻辑导致代码臃肿。

解决方案：

- 策略模式：定义 `IHouseFilter`（区域/租金筛选）和 `ISortStrategy`（价格/距离排序）；
- 工厂模式： `SortStrategyFactory` 根据参数动态选择排序策略；
- 优势：新增筛选条件只需实现新类，无需修改主逻辑。

代码块

```
1 // 策略模式示例
2 List<HouseDTO> filtered = houseList.stream()
3     .filter(house -> filterMap.values().stream()
4         .allMatch(filter -> filter.apply(house, reqDTO))) // 组合多个筛选器
```

```
5         .collect(Collectors.toList());
```

9. 在房源搜索的筛选和排序功能中，V1版本直接硬编码逻辑，而V2版本引入策略模式+工厂模式。为什么V1版本违反单一职责原则？举例说明其维护成本高的场景。

V1的 `filterHouseV1()` 方法同时承担：

- 区域/租金/户型等多维度筛选
- 价格/距离等多规则排序
- 分页逻辑处理
 - 示例维护场景：当新增“宠物友好”筛选时，需直接修改该方法，可能影响已有逻辑（如误删排序代码）。

10. 策略模式如何解决筛选条件的扩展问题？新增一个“是否地铁房”筛选条件需改动哪些代码？

扩展步骤：

- 新增 `SubwayFilter` 实现 `IHouseFilter`
- 在请求DTO中添加字段 `Boolean requireSubway`

代码块

```
1  @Componentpublic
2  class SubwayFilter implements IHouseFilter {
3      @Override
4      public Boolean filter(HouseDTO house, SearchHouseListReqDTO req) {
5          // 从房源标签判断是否近地铁
6          return req.getRequireSubway() == null
7              || house.getTags().contains("near_subway");
8      }
9  }
```

无需修改主流程，新增类自动被 `houseFilterMap` 收集

11. 为什么用 RabbitMQ 监听用户信息修改？如何保证消息不丢失？

用户信息更新需同步刷新多个房源缓存，MQ解耦业务逻辑。

防丢失方案：

- 生产者确认：开启Publisher Confirm，确保消息到达Broker；
- 持久化：队列/消息均持久化；
- 消费者ACK：手动ACK，处理成功后才删除消息；
- 死信队列：重试多次失败后转入死信队列人工干预。

12. 解释Nacos在项目中的双重角色（注册中心+配置中心）

注册中心：

- 服务注册：各微服务启动时注册IP+端口；
- 服务发现：Gateway通过Nacos定位微服务实例。

配置中心：

- 集中管理：数据库连接、线程池参数等配置；
- 动态更新：修改配置后实时推送给服务，无需重启。

13. 如何实现“城市维度房源列表缓存”的更新？

- 缓存结构：
Redis Key = `house:list:{cityId}`，Value = 房源ID列表（List类型）。
- 更新时机：
 - 手动更新：管理员触发 `/admin/house/refresh` 接口；
 - 自动更新：房源增删时，删除旧城市ID列表并添加到新城市列表。
- 注意：缓存不过期，依赖人工刷新保证一致性。

14. 为什么用Redisson而不是Redis SETNX实现分布式锁？

SETNX 缺陷：

- 需自行处理锁续期（业务执行时间长导致锁过期）；
- 释放锁时需Lua脚本保证原子性（避免误删其他线程锁）。

Redisson 优势：

- Watchdog 机制：后台线程自动续期（默认30s）；
- 可重入锁：支持同一线程多次加锁；
- API封装：`lock()` / `unlock()` 方法开箱即用。

15. 用户定位功能的技术实现？如何选择地图服务？

流程：前端获取经纬度 → 调用腾讯地图API逆地址解析 → 返回城市/区域信息。

选型依据：

- 成本：腾讯地图开发者免费额度较高；
- 精度：国内定位精度优于Google Map；
- 合规性：符合国内地理信息安全要求。

16. 咨询消息的持久化方案如何优化？

分库分表：按 `user_id` 哈希分库，按月份分表（如 `chat_msg_2023_10`）；

读写分离：

- 写操作：主库（高并发下队列缓冲）；
- 读操作：从库 + Redis缓存最近10条消息；

冷热分离：超过3个月的消息转存至Elasticsearch。

17. Git 分支模型如何支持多版本发布？

采用 GitFlow模型：

- `master`：生产环境代码（只读）；
- `develop`：集成测试分支；
- `feature/*`：功能开发分支（如 `feature/chat-module`）；
- 流程：
`feature` → `develop`（测试） → `master`（发布） → 打Tag（v1.0.0）。

18. 如果房源详情页QPS达到1万，如何优化？

分层优化：



具体措施：

- Redis 缓存命中率提升：预热热点房源；

- MySQL：添加读写分离 + 二级缓存（Caffeine）；
- 限流：Sentinel对高频查询 `/detail` 接口限流。